

Project One: **Solve the River Crossing Puzzle**

By: Azhar Abbas Syed and Ariana Sookoo

Q1.2 - DFS and BFS comparison

Using the initial state 3, 3, 0, 0, L:

1. Report the solution path and number of node expansions for DFS and BFS.

	DFS	BFS
Solution Path	(3, 3, 0, 0, 'L') (2, 2, 1, 1, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (1, 1, 2, 2, 'L') (0, 0, 3, 3, 'R')	(3, 3, 0, 0, 'L') (2, 2, 1, 1, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (1, 1, 2, 2, 'L') (0, 0, 3, 3, 'R')
Total Cost	11	11
Number of Node Expansions	13	14

2. Write a short free-form explanation (1–2 paragraphs) comparing DFS and BFS in the context of this problem:

- **What trade-offs do you observe?**

DFS explores one path deeply before going to the next choice, while BFS explores one level at a time. DFS usually uses less memory, while BFS usually stores more states. But both found an 11- crossing solution here.

- **Why does BFS guarantee an optimal solution (if one exists), while DFS may fail to do so?**

BFS guarantees the shortest solution because every crossing costs 1 in this problem, and BFS checks the shorter path first. DFS may find a longer solution because it follows a single path to the end without checking if there is a shorter one.

- **What do the node expansion counts tell you about efficiency vs. completeness?**

DFS expanded 13 nodes, while BFS expanded 14 in this test case. So DFS did less search computation in this run. But this does not mean DFS is always more efficient. Node expansion measures search effort, while completeness means the algorithm is guaranteed to find a solution if it exists. Here the state space is finite, and both searches track visited states, so both are complete.

Q2.2 - Uniform Cost Search (UCS) with Non-Uniform Action Cost

Using initial state 3, 3, 0, 0, L:

1. Run UCS with **Cost Model A** and **Cost Model B**, report:

- Solution path
- Total cost
- Number of node expansions

	Cost Model A	Cost Model B
Solution Path	(3, 3, 0, 0, 'L') (3, 1, 0, 2, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (0, 2, 3, 1, 'L') (0, 0, 3, 3, 'R')	(3, 3, 0, 0, 'L') (2, 2, 1, 1, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (1, 1, 2, 2, 'L') (0, 0, 3, 3, 'R')
Total Cost	23	17
Number of Node Expansions	14	14

1. Compare with BFS:

- **Does UCS choose a different sequence of actions?**

The UCS algorithm chooses a different sequence of actions in Cost Model A than BFS. They diverge in the second state and in the eleventh state. Where BFS transfers one missionary and one cannibal over to the right side (2, 2, 1, 1, 'R'), UCS transfers two cannibals (3, 1, 0, 2, 'R'). Then, in the eleventh state, BFS sends a missionary to the left side (1, 1, 2, 2, 'L'), whereas UCS sends one cannibal to the left side (0, 2, 3, 1, 'L'). From this, we can see that UCS avoids putting missionaries on the boat when a cannibal can make the trip due to the cost (missionaries = 2 and cannibals = 1). In both changes, one missionary is switched with a cannibal. However, there is no difference between Cost Model B and the path stated by BFS.

- **How does total cost (UCS) relate to path length (BFS)?**

For this comparison, all paths are the same length; therefore, UCS prioritises the less costly path. For Cost Model A, BFS would cost 27, while UCS would find one

with a cost of 23. Then, for Cost Model B, the cheapest and shortest path are the same.

- **Which cost model leads to more/fewer node expansions?**

Both models lead to 14 node expansions; therefore, neither.

2. **In 1–2 paragraphs, explain why UCS is necessary with non-uniform costs, and discuss the trade-off between least cost and fewest steps.**

UCS is necessary with non-uniform costs as it guarantees finding the minimum total cost path. This is different from BFS, which only guarantees the shortest path when each action costs the same amount. The trade-off occurs based on whether the shortest path or least cost is prioritised. This becomes apparent in situations where paths that are shorter in length may cost more, paths that are equal but differ in cost, and vice versa for both scenarios. UCS is important as it expands the node with the lowest cumulative cost $g(n)$ instead of the shallowest node. Therefore, this guarantees that the first goal it removes from the queue is a least-cost solution as long as all action costs are positive. With Cost Model A, this is important as the BFS solution costs 27 and UCS finds an equally short path which costs 23. Then, with Cost Model B, the cost of the path depended on the length since the boat trip alternated direction. Since UCS and BFS agree, it shows that UCS behaves like BFS when costs do not favour any particular action.

Q3.2 The Writing Component

Part A — Admissibility Proofs

For each heuristic (h_1 , h_2):

- **Prove or disprove that it is admissible.**
- **If admissible, explain why it never overestimates.**
- **If not, provide a counterexample**

Heuristic 1 is admissible because everyone remaining on the left must cross right at least one time. Each missionary costs 2, and each cannibal costs 1 to transport. The heuristic counts only these necessary crossings. Therefore, any solution must cost at least $2M_{\text{left}} + C_{\text{left}}$. A return trip may increase the cost, but this heuristic doesn't include it, so it will only underestimate the true remaining cost.

Heuristic 2 is also admissible because you are dividing the value of heuristic 1 by three and rounding up, which will never make it larger than heuristic 1. Since heuristic 1 never overestimates, heuristic 2 also cannot overestimate.

Part B — Implementation and Comparison

1. Implement A* with h_1 and h_2 .
2. Run both on the initial state 3, 3, 0, 0, L
3. Record results in a table:

Heuristic	Solution path brief	Total cost	Node expansion	Same cost as UCS?
h1	(3, 3, 0, 0, 'L') (3, 1, 0, 2, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (0, 2, 3, 1, 'L') (0, 0, 3, 3, 'R')	23	13	Yes
h2	(3, 3, 0, 0, 'L') (3, 1, 0, 2, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (0, 2, 3, 1, 'L') (0, 0, 3, 3, 'R')	23	14	Yes
h3	(3, 3, 0, 0, 'L') (3, 1, 0, 2, 'R') (3, 2, 0, 1, 'L') (3, 0, 0, 3, 'R') (3, 1, 0, 2, 'L') (1, 1, 2, 2, 'R') (2, 2, 1, 1, 'L') (0, 2, 3, 1, 'R') (0, 3, 3, 0, 'L') (0, 1, 3, 2, 'R') (0, 2, 3, 1, 'L') (0, 0, 3, 3, 'R')	23	13	Yes

4. **Compare node expansions: which heuristic was more efficient, and why?**

h_1 is more efficient, with 13 node expansions compared with 14 for h_2 . h_2 needs the same number as UCS, which uses no heuristic. The closer a heuristic is to the true cost, the fewer nodes A* has to explore. The true cost is 23 at the start state, and using the heuristic formulas provided, h_1 estimates 9 and h_2 estimates 3 due to the division. h_1 provides a stronger estimate and dominates h_2 while remaining admissible. This

difference is minor, however, with only one node expansion difference due to how small the puzzle state space is.

Part C — Designing a Tighter Heuristic

Propose your own heuristic h_3 that is admissible and which dominates h_1 or h_2 . Note, we are not looking for the $\max(h_1, h_2)$ heuristic. We want you to get creative with your heuristic.

- **Define it clearly.**

Heuristic 3 — Remaining Weight and Boat Position Requirements

Remaining People = $M_{\text{left}} + C_{\text{left}}$

$h_3(s) = 2M_{\text{left}} + C_{\text{left}} + 2$

(if the boat is on the right side and Remaining People > 0).

Else, $h_3(s) = 2M_{\text{left}} + C_{\text{left}}$

In h_1 , it's assumed that all people on the left can cross. But if the boat is on the right side, no one can leave until the boat returns. That return trip has to cost something, and the person who brings the boat back has to cross the river again, which also costs something. Since the extra term is always 0 or 2, $h_3(s) \geq h_1(s) \geq h_2(s)$ for every state, so h_3 dominates both h_1 and h_2 .

- **Argue why it is admissible.**

Under Cost Model A, carrying a person costs their weight ($M = 2$, $C = 1$). Therefore, transferring everyone to the right side of the river costs at least $2M_{\text{left}} + C_{\text{left}} = h_1(s)$. If the boat is on the right side and the number of people remaining on the left is more than zero, no one can cross the river until someone returns with the boat. Since the boat carries at least one person, the cost is at least 1. Then they have to cross the river again, which costs an additional 1. These extra trips aren't accounted for in the first heuristic, so the true cost is at least $h_1 + 2$, which does not overestimate. The condition that remaining people > 0 is needed because at the goal state ($0, 0, 3, 3, R$) the true cost is 0, but without it h_3 would give 2, which overestimates and violates admissibility.

- **Implement h_3 in A* and compare node expansions against h_1 and h_2 by adding a third row in the table above (this completed table should be added to proj-1-submission.pdf (the written file that you will submit with your project))**

When comparing, it was found that both h_3 and h_1 have 13 node expansions, whereas h_2 had one more node. At the start state, the boat is on the left, so $h_3 = h_1 = 9$. The additional 2 bonus only applies once the boat is on the right, and in this small puzzle it doesn't change which node A* expands next, so h_3 cannot improve on h_1 here even though it is at least as tight in every state.

Q3.3 Extra Credit — Consistency Proofs

For each heuristic (h_1 , h_2 , and your h_3):

- Prove or disprove that it is consistent.
- Recall: h is consistent if for every edge $s \rightarrow s'$ with cost c :

$$h(s) \leq c + h(s')$$

Heuristic 1 (h_1) is consistent. When the boat moves left to right, h_1 decreases by exactly the move's cost, and when it moves right to left, h_1 increases, so $h_1(s) \leq c + h_1(s')$ holds for every move.

Heuristic 2 (h_2) is also consistent, as dividing the weight by 3 makes it change more slowly than the actual move cost. When the boat moves left to right, h_2 decreases by at most the cost of the move, and when the boat moves right to left, the remaining weight increases, so h_2 can only stay the same or increase. Since h_2 cannot decrease faster than the cost of an action, it is consistent for every legal move.

Heuristic 3 (h_3) is consistent, similar to the other heuristics. Let a move carry weight w , so its cost is $c = w$. When the boat moves from left to right, h_1 decreases by exactly w , and the additional 2 can only switch on. So h_3 drops by at most c . Then, when the boat moves from right to left, h_1 rises by w and the additional 2 switches off, so h_3 drops by at most $2 - w \leq 1 \leq c$. In both cases, $h_3(s) \leq c + h_3(s')$, therefore h_3 is also consistent.